

Uma janela clara para um fluxo que já existe.

Skill para transformar automações Windows existentes em apps WPF portáteis, mantendo a regra de negócio nos scripts oficiais e trazendo contexto para a operação.

A pergunta central

O aplicativo não precisa saber mais do que o fluxo. Precisa saber o bastante para mostrar o que aconteceu, o que ficou pendente e qual é a próxima ação segura.

Fluxo em camadas

1	Inventário transitivo Mapeie chamadas, entradas, saídas, dependências, efeitos, locks e arquivos protegidos.	2	Contrato de resultado Defina run_id, métricas, exceções, estado e próxima ação em JSON UTF-8 fresco.	3	Interface WPF Separe ApplInterface, worker, parser e rodador oficial; mantenha a regra onde ela já está.	4	QA observável Teste fixtures e a janela real: foco, teclado, rolagem, redimensionamento e estados.
----------	--	----------	--	----------	--	----------	--

O que melhora na prática

Regra preservada O app envolve os rodadores existentes. Reescrever a regra de negócio exige pedido e escopo próprios.	Resultado legível Contagens, cardinalidades, exceções e próxima ação aparecem antes do ruído técnico.	Portabilidade Caminhos relativos, pré-requisitos explícitos e preparação separada ajudam a levar o app a outro computador.
---	---	--

Contratos neutros: wpf.app.result.v1 para o resultado e wpf.flow.inventory.v1 para o inventário.

Quando usar e como manter o limite

A abordagem é útil quando uma automação Windows já funciona em algum grau, mas sua operação precisa de uma janela portátil, uma leitura de resultado e uma trilha de decisão mais clara. A dependência \$cacau é verificada e necessária para separar leitura, mudança e revisão.

Situações de uso

Fluxo com vários componentes

Quando R, PowerShell, Excel, e-mail, navegador ou rede aparecem em uma cadeia transitiva que precisa ser inventariada.

Equipe em outro computador

Quando pré-requisitos, caminhos e permissões precisam ser verificados sem executar o fluxo por acidente.

Resultado com pendência

Quando encerrar o processo não significa que todos os itens foram confirmados, importados, enviados ou verificados.

Ação de risco

Quando existe pagamento, envio, upload, importação ou limpeza: separar confirmação, efeito e reconciliação.

Segurança visível

SUCESSO

Critérios obrigatórios confirmados.

OK_COM_PENDENCIAS

Terminou, mas exige decisão.

ERRO

Falhou sem confirmação suficiente.

BLOQUEADO

Não deve começar ou repetir.

Exemplo de instalação

```
$skill-installer Instale a skill Scripts em APP WPF:  
https://github.com/cacauzuxa/scripts-em-app-wpf/tree/main/app
```

Depois, invoque \$app. A dependência \$cacau será verificada e é necessária para separar leitura, mudança e revisão. Uma instalação bem-sucedida não prova homologação: ainda são necessários inventário, contrato, testes, QA em janela real e autorização para efeitos externos.

Limites honestos

Não trate exit code zero, arquivo criado, processo encerrado, barra em 100% ou fixture como prova de negócio. Preserve logs técnicos sem expor credenciais ou dados locais e mantenha a repetição bloqueada quando a confirmação externa for incerta.